

RAPPORT TEST

Rapport de test

Table des matières

Rapport Complet de Test et Validation – Application Flask.....	2
1. Introduction.....	2
2. Objectifs du Test Logiciel.....	3
3. Concepts Fondamentaux des Tests (Module 1).....	3
3.1 Vérification vs Validation.....	3
3.2 Assurance Qualité (QA) vs Test Logiciel.....	3
3.3 Tests Manuels vs Tests Automatisés	3
3.4 Cycle de vie des tests	3
4. Stratégies et Techniques de Test (Module 2).....	3
4.1 Test Boîte Blanche.....	4
4.2 Test Boîte Noire.....	4
4.3 Conception des Cas de Test	4
5. Méthodologie de Test – Application Flask.....	5
5.1 Environnement de Test	5
5.2 Types de Tests Réalisés	5
6. Résultats des Tests et Analyse	5
7. Conclusion	6

Rapport Complet de Test et Validation – Application Flask

1. Introduction

Ce rapport présente l'ensemble du processus de **test et validation logicielle** appliqué à une application **Flask** de gestion des compétitions sportives.

Il combine les connaissances théoriques issues des modules de formation (“Test et Validation”) et leur application pratique à travers des **tests automatisés avec Pytest**.

L’objectif principal est de garantir la **qualité**, la **fiabilité**, et la **conformité fonctionnelle** du système avant sa mise en production.

2. Objectifs du Test Logiciel

Les tests logiciels visent à :

- Détecter les bogues avant la mise en production.
- Améliorer la fiabilité et la robustesse du système.
- Garantir la conformité aux spécifications.
- Réduire les coûts de maintenance.

3. Concepts Fondamentaux des Tests (Module 1)

3.1 Vérification vs Validation

- **Vérification** : S’assurer que le logiciel est construit correctement.
- **Validation** : S’assurer que le logiciel répond aux besoins utilisateurs. Exemple :

Vérifier qu’une fonction `calculate_total()` retourne une somme correcte (vérification) et que le montant affiché à l’écran correspond aux attentes du client (validation).

3.2 Assurance Qualité (QA) vs Test Logiciel

- **QA** : Processus global garantissant les normes de qualité (ex : méthode Agile, revue de code).
- **Test logiciel** : Activité ciblée pour détecter les défauts avant déploiement.

3.3 Tests Manuels vs Tests Automatisés

Type	Avantages	Inconvénients
Manuels	Utile pour les tests exploratoires et UX	Lents, risque d’erreur humaine
Automatisés	Rapides, répétables, intégrables en CI/CD	Temps initial d’implémentation

3.4 Cycle de vie des tests

- **Modèle en V** : chaque phase de conception a son test associé.
- **Agile** : tests continus (TDD).
- **DevOps** : intégration continue via pipelines automatisés.

4. Stratégies et Techniques de Test (Module 2)

4.1 Test Boîte Blanche

Basé sur le **code interne**. Permet d'assurer une couverture complète :

- **Instructions**
- **Branches (if/else)**
- **Conditions logiques** □ **Chemins d'exécution**

4.2 Test Boîte Noire

Basé sur les **entrées/sorties** sans connaître le code :

- **Partition d'équivalence** (groupes de valeurs similaires)
- **Valeurs limites** (ex : 0, -1, max, min)
- **Tables de décision** et **transitions d'état**

4.3 Conception des Cas de Test

Un bon cas de test contient :

- Identifiant, description, entrées, conditions préalables, résultat attendu, statut.

ID	Cas de test	Entrée	Sorties	Statut
TC01	Vérifier l'accès à la page	GET /	200 OK	✓
TC02	Vérifier la connexion avec email valide	email=kodjoandre56@gmail.com	Page compétitions	✓
TC03	Verifier la connexion avec email non valide	Email=kouakou55@gmail.com	Erreur	✗
TC04	Reserver place > 12	Nombre de places	Erreur	✗
TC05	Reserve place <= 12	Nombre de places	200 ok	✓
TC06	Points<places	places	Erreur	✗

4.4 Priorisation par Risque

Fonctionnalité	Probabilité	Impact	Priorité
Connexion	Élevée	Critique	Haute
Réservation	Moyenne	Très critique	Haute
Page d'accueil	Faible	Mineur	Moyenne
Point < places	Eleve	Critique	Haute

5. Méthodologie de Test – Application Flask

5.1 Environnement de Test

Les tests ont été exécutés avec **Pytest**, à l'aide d'un **client Flask de test** simulant les requêtes HTTP.

```
@pytest.fixture def
client():
    app.config['TESTING'] = True
    with app.test_client() as client:
        yield client
```

Ce client permet d'isoler chaque test dans un environnement contrôlé sans lancer le serveur.

5.2 Types de Tests Réalisés

- **Unitaires** → Fonctions isolées (logique métier).
- **Intégration** → Interaction entre routes et base de données.
- **Boîte noire** → Tests sur comportements attendus.
- **Boîte blanche** → Vérification des conditions internes.

6. Résultats des Tests et Analyse

Nom du test	Description	Résultat attendu	Statut
test_dashboard_page	Accès à “/”	200 OK + “Club”	✓
test_index_page	Accès à “/index”	200 OK	✓

test_logout_redirect	Déconnexion redirige vers “/”	302 Redirect	✓
test_welcome_page	Accès à “/welcome”	200 OK	✓
test_show_summary_valid_email	Connexion valide	200 OK + compétitions	✓
test_show_summary_invalid_email	Email invalide	400 Bad Request	✓
test_purchase_success	Achat correct	“Booking complete”	✓
test_purchase_not_enough_points	Points insuffisants	Message d’erreur	✓
test_purchase_not_enough_places	Places insuffisantes	Message “Not enough places”	✓
test_purchase_past_competition	Compétition passée	Message “past competitions”	✓
test_purchase_more_than_12	>12 places demandées	Message “more than 12”	✓

Bilan :

- 11 tests exécutés
- 11 réussis

7. Conclusion

Ce projet démontre la **complémentarité entre théorie et pratique** :

- Les notions apprises (vérification, validation, boîtes blanche/noire) ont été appliquées efficacement.
- Les tests assurent la conformité, la fiabilité et la maintenabilité du système Flask.
- L’approche adoptée s’inscrit dans le **modèle en V** et la philosophie **DevOps**, assurant une validation continue.